

ARTIFICIAL INTELLIGENCE (CSA1703)

Assessment Tool – 2

CO2: Experiment Search and Game-Solving Techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta Pruning with Heuristic Functions for Solving Complex AI Problems Efficiently

Question 1: AI-Powered Drone Navigation

Problem

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic, with blocked roads, changing weather conditions, and variable travel costs. The drone must quickly find the safest and shortest route.

i) Greedy Best-First Search (GBFS)

Working

- Uses only the heuristic value $h(n)$.
- Chooses the node that appears closest to the destination.

Strengths

- Fast decision making.
- Easy to implement.
- Suitable for emergency situations.

Weaknesses

- Ignores actual path cost.
 - May select unsafe or non-optimal routes.
-

ii) A* Search

Working

Uses the evaluation function:

$$f(n) = g(n) + h(n)$$

Where:

- $g(n)$ = Actual path cost.
- $h(n)$ = Estimated distance to the goal.

Advantages

- Finds the optimal path.

- Considers both travel cost and heuristic.
 - Suitable for dynamic environments.
-

iii) Impact of Heuristic Accuracy

Accurate Heuristic

- GBFS reaches the goal quickly.
- A* finds the optimal path with fewer node expansions.

Poor Heuristic

- GBFS may choose incorrect paths.
 - A* becomes slower but still produces a better solution.
-

iv) Effect of Dynamic Changes

- Blocked roads require route replanning.
 - Weather changes increase travel cost.
 - GBFS may continue on a poor route.
 - A* adapts better because it considers both path cost and heuristic.
-

v) Recommended Algorithm

A* Search

Reason

- Produces the safest and shortest route.
 - Balances speed and optimality.
 - Suitable for real-time emergency medicine delivery.
-

Conclusion

A* Search is the most suitable algorithm because it finds the optimal path while adapting to changing environments, ensuring fast, safe, and reliable medicine delivery during emergencies.

Question 2: Smart City Traffic Optimization

Problem

A smart city uses an AI system to optimize traffic signal timings to reduce waiting time, fuel consumption, and traffic congestion. The search space is very large, and traffic conditions change continuously.

i) Optimization Problem

Objective

- Minimize waiting time.
- Reduce fuel consumption.
- Reduce traffic congestion.

Reason

Traditional search is inefficient because the search space is huge and traffic conditions change dynamically.

ii) Hill Climbing

Working

- Starts with an initial solution.
- Moves to a better neighboring solution until no further improvement is possible.

Advantages

- Simple and fast.
- Requires less memory.

Limitations

- Can get stuck at local maxima.
 - Cannot always find the global optimum.
-

iii) Local Maxima, Plateaus and Ridges

- **Local Maxima:** Stops at a good but not the best solution.
 - **Plateaus:** No improvement is possible because neighboring states have equal values.
 - **Ridges:** Progress is slow due to narrow improvement paths.
-

iv) Simulated Annealing / Genetic Algorithm

Simulated Annealing

- Accepts worse solutions occasionally to escape local maxima.

Genetic Algorithm

- Uses selection, crossover, and mutation to generate better traffic signal plans.
-

v) Recommended Approach

Genetic Algorithm

Reason

- Handles large search spaces.
 - Adapts to changing traffic conditions.
 - Produces better optimization results.
-

Conclusion

Genetic Algorithm is the most suitable approach because it provides scalable and adaptive optimization for smart city traffic management.

Question 3: Mars Rover Online Search Agent

Problem

An autonomous Mars rover must explore unknown terrain, avoid hazards, collect samples, and make decisions with limited sensing capability and no complete map.

i) Need for Online Search

The rover has no complete map of Mars. It must make decisions while exploring the environment.

ii) Challenges

- Partial observability.
 - Unknown obstacles.
 - Dynamic environment.
 - Limited sensor information.
-

iii) Internal Model

The rover:

- Stores explored locations.
 - Updates the map continuously.
 - Uses sensor information for future navigation.
-

iv) Comparison

Online Search

- Learns while moving.

- Suitable for unknown environments.

Uninformed Search

- No heuristic guidance.

Informed Search

- Uses heuristics but usually requires a known map.
-

v) Recommended Approach

Online Search Agent

Reason

- Adapts to uncertainty.
 - Updates knowledge continuously.
 - Ensures safe navigation.
-

Conclusion

An Online Search Agent is the best choice because it allows the rover to explore safely and make intelligent decisions in an unknown environment.

Question 4: University Exam Timetable using CSP

Problem

A university wants to generate an exam timetable while satisfying constraints such as no overlapping exams, limited halls, faculty availability, and student requirements.

i) Constraint Satisfaction Problem (CSP)

Variables

- Exams.

Domains

- Available dates and time slots.

Constraints

- No overlapping exams.
- Limited exam halls.
- Faculty availability.
- Subject order.
- Special accommodations.

ii) Backtracking Search

- Assigns one exam at a time.
 - If a constraint is violated, returns to the previous step and tries another option.
-

iii) Constraint Propagation

Forward Checking

- Removes invalid choices early.
 - Reduces unnecessary search.
 - Improves efficiency.
-

iv) Real-World Challenges

- Large number of courses.
 - Large number of students.
 - Multiple scheduling constraints.
-

v) Recommended Strategy

CSP with Backtracking and Forward Checking

Reason

- Satisfies all constraints.
 - Reduces computation time.
 - Produces an efficient timetable.
-

Conclusion

Constraint Satisfaction Problem with Backtracking and Forward Checking is the best approach for generating an efficient and conflict-free exam timetable.

Question 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)

Problem

A gaming company is developing an AI opponent for a real-time strategy game. The AI must make quick and intelligent decisions despite a very large game tree.

i) Minimax Algorithm

Working

- Assumes both players make the best possible moves.
 - Selects the move with the highest guaranteed score.
-

ii) Computational Challenges

- Large game tree.
 - High memory usage.
 - Longer computation time.
-

iii) Alpha-Beta Pruning

Working

- Removes branches that cannot affect the final decision.
 - Reduces the number of nodes explored.
 - Improves search speed without affecting the result.
-

iv) Evaluation Function

Factors Considered

- Player score.
 - Army strength.
 - Resources.
 - Map control.
 - Distance from the opponent.
-

v) Recommended Strategy

Minimax with Alpha-Beta Pruning

Reason

- Produces strong decisions.
 - Reduces computation time.
 - Suitable for real-time games.
-

Conclusion

Minimax with Alpha-Beta Pruning is the most suitable strategy because it balances optimal decision-making with fast execution, making it ideal for real-time strategy games.

